

AI Prompts Used During Development

Document: Candidate Challenge Submission — Section 6

Project: AI Study Companion (Production Google Gemini & Supabase pgvector Architecture)

Author / Candidate: Engineering Challenge Submission

Date: September 2026

1. Executive Overview

This document catalogs the actual, high-impact prompts utilized across the development lifecycle of the **AI Study Companion**. Each prompt reflects a real engineering milestone—ranging from initial architectural decomposition and schema modeling to UI/UX styling, resilient coding, RAG grounding, adaptive quiz rubric generation, and critical production debugging.

2. Architecture & Planning Prompts

Prompt 2.1: PRD Decomposition & Closed Cognitive Learning Loop

Tool Used: ChatGPT (GPT-4o)

Context: Initial project kickoff after receiving PRD v3.0.

Goal: Break down the disconnected CRUD model into an active, closed-loop learning architecture.

```
You are an expert Principal AI Systems Architect.
I am building the "AI Study Companion" for an AI Engineer Candidate Challenge based on PRD v3.0.

The core requirements are:
1. Spaces & Projects multi-tenant hierarchy with strict user isolation.
2. Ingestion of PDFs into semantic chunks with 1536-dimensional embeddings and concept extraction.
3. Socratic AI Tutor with streaming and page-level citations.
4. Deterministic Insufficient Evidence Guard (zero hallucinations, reject off-topic queries like cooking r
5. Multi-signal adaptive assessment (MCQ and open-ended) based on 7 learner signals.
6. Open-ended question evaluation using structured rubrics (accuracy, conceptual depth, missing concepts).
7. Deterministic concept mastery evolution: New Mastery = 0.6 * Prev + 0.4 * Recent.
8. Growth Analytics and Prioritized Recommendations.
9. Admin telemetry dashboard (latency, input/output tokens, cost in USD).

Help me architect the complete system:
- Outline the closed cognitive learning loop lifecycle.
- Propose the 16 relational database tables needed in PostgreSQL with pgvector.
- Define the event-driven background job workflows using Inngest.
- Provide a C4 Level 1 and Level 2 container architecture diagram.
Keep it strictly technical, rigorous, and production-grade.
```

Prompt 2.2: Multi-Signal Concept Selection Algorithm Formulation

Tool Used: ChatGPT (GPT-4o)

Context: Designing the adaptive quiz generation logic in `lib/learning/adaptive-selection.ts`.

Goal: Translate subjective learning signals into an objective, explainable mathematical score.

I need to formulate a deterministic, explainable mathematical priority scoring formula for selecting concepts.

I have 7 signals mandated by the PRD:

1. Concept Mastery / Weakness (lower mastery = higher priority)
2. Recorded Mistakes (repeated errors = immediate urgency)
3. Recent Assessment Performance (low recent score = boost; high recent score = reduce redundancy)
4. Target Difficulty (dynamically calibrated 1-5)
5. Question History (avoid repeating recently assessed concepts if unassessed alternatives exist)
6. Recent Learning Activity (reinforce concepts discussed in recent tutor chat sessions)
7. Staleness (concepts unassessed for > 3 days gain priority)

Please design:

1. A weighted additive priority formula: $Priority = W_{weakness} + W_{mistake} + W_{staleness} + W_{activity} + B_{target}$
2. Explicit numerical caps and ranges for each weight (e.g. Weakness: 0-50 pts, Mistakes: 0-30 pts, Staleness: 0-100 pts)
3. A deterministic rule for mapping learner mastery and mistake frequency to an integer difficulty level
4. Edge-case handling for brand-new concepts with zero assessment history.

3. UI / UX Design & Presentation Prompts

Prompt 3.1: Glassmorphism Dark Mode Design System with Tailwind CSS v4

Tool Used: Claude 3.7 Sonnet

Context: Setting up the global styling and visual theme in `app/globals.css` and `components/ui/`.

Goal: Create an ultra-premium, dark-mode-first aesthetic with rich micro-animations.

You are an elite Frontend UI/UX Engineer specialized in Tailwind CSS v4 and React 19.

I am styling the "AI Study Companion" web application.

I need a cohesive, modern dark-mode aesthetic:

- Deep slate/navy background canvas (#0b0f19 to #020617) with subtle radial glowing gradients.
- Glassmorphism card surfaces with subtle borders (rgba(255, 255, 255, 0.08)), backdrop blur, and smooth hovers.
- Accent colors: Electric Indigo / Cobalt Blue (#3b82f6 to #6366f1) for primary actions, Emerald Green (#22c55e) for success states.
- Modern typography pairing: Inter for UI body, JetBrains Mono for code excerpts and citations.

Generate the Tailwind CSS utility tokens and component patterns for:

1. Socratic Chat bubble (user message vs tutor message with source citation badges).
2. Adaptive Quiz Card with timer countdown, difficulty badge (1-5 stars), and animated option buttons.
3. Mastery Progress Ring / Bar with trend indicator (+/- change).

Ensure WCAG AA contrast compliance throughout.

Prompt 3.2: Recharts Growth & Telemetry Visualizations

Tool Used: Claude 3.7 Sonnet

Context: Implementing the `/growth` and `/admin` visual analytics pages.

Goal: Build responsive, interactive data charts for concept mastery and AI token consumption.

Create two reusable React components using Recharts and Tailwind CSS:

1. ``MasteryRadarChart.tsx``:
 - Visualizes concept mastery scores (0-100) across 5-8 topics in a project.
 - Shows both "Current Mastery" and "Baseline / Previous Score" as dual polygon layers with semi-transparent fill.
 - Includes custom tooltip showing concept name, score, and trend status (IMPROVING, STABLE, NEEDS_ATTENTION).
2. ``AdminTelemetryDashboard.tsx``:
 - Displays real-time operational metrics for AI model usage.
 - Metric cards: Total Requests, Total Input Tokens, Total Output Tokens, Estimated Spend (\$ USD), and Average Latency.
 - Area chart showing daily token consumption grouped by feature (tutor, quiz, assessment, extraction).
 - Real-time audit log stream showing user registrations, PDF uploads, and quiz completions.

4. Coding & Resilient Implementation Prompts

Prompt 4.1: Google Gemini Resilient Provider Abstraction

Tool Used: Claude 3.7 Sonnet

Context: Implementing `lib/ai/gemini.ts` using the new `@google/genai` SDK.

Goal: Build an enterprise-grade AI provider with task-aware routing, exponential backoff, and model failovers.

Write a production-grade TypeScript implementation for ``GeminiProvider`` implementing the ``AIProvider`` interface.

Requirements:

1. Environment Configuration:
 - Read ``GEMINI_API_KEY``, ``TUTOR_MODEL`` (default: `gemini-2.5-flash`), ``FAST_MODEL`` (default: `gemini-2.5-flash`).
2. Methods to implement:
 - ``generateText(prompt, options)``: supports system instructions, temperature, maxTokens, and structured outputs.
 - ``streamText(prompt, options)``: returns an async iterable stream of text chunks.
 - ``generateEmbedding(text)``: invokes `gemini-embedding-001` with `outputDimensionality: 1536`.
3. Resilience Engineering:
 - Implement automatic retry with exponential backoff and randomized jitter for HTTP 429 and 503 errors.
 - Implement dynamic cross-model fallback: if ``gemini-2.5-flash`` fails repeatedly, transparently fallback to ``FAST_MODEL``.
4. Usage Telemetry:
 - Log `latency_ms`, `input_tokens`, `output_tokens`, `model`, and calculated USD cost to the ``ai_usage_logs`` table.

5. RAG, Grounding & AI Tutor Prompts

Prompt 5.1: Socratic Pedagogical Tutor System Prompt

Tool Used: Claude 3.7 Sonnet

Context: The system prompt embedded in `lib/ai/tutor.ts` for the live conversational tutor.

Goal: Enforce Socratic methodology, grounded citations, and zero-hallucination discipline.

You are the Socratic AI Study Companion for this course project.
Your mission is to guide the student toward deep understanding using active inquiry rather than passive learning.

STRICT PEDAGOGICAL RULES:

1. Never provide the direct final answer immediately unless the student has successfully reasoned through the problem.
2. Ask probing, diagnostic questions that guide the learner to discover the answer themselves.
3. If the student makes a conceptual error, point out the contradiction gently and ask them to re-evaluate their reasoning.
4. Ground every explanation strictly in the retrieved course context provided below.
5. Provide exact citations in the format: [Page X, "quoted phrase from text"].

INSUFFICIENT EVIDENCE RULE:

- If the retrieved context does not contain sufficient factual evidence to answer the student's question, respond with: "I cannot find sufficient evidence in your uploaded course materials to answer this question."
- Do NOT use general world knowledge to answer questions outside the uploaded materials.
- Under no circumstances answer off-topic queries (such as cooking recipes, pop culture, or unrelated trivia).

Retrieved Course Context:

```
{{retrieved_chunks_with_page_numbers}}
```

Prompt 5.2: Insufficient Evidence Guard & Off-Topic Query Refusal

Tool Used: Claude 3.7 Sonnet / Perplexity AI

Context: Refining `lib/rag/grounding.ts` and similarity threshold gating.

Goal: Validate that off-topic queries (e.g. "How to cook chicken biryani?") are rejected with 0 citations.

How do I implement a deterministic two-stage Insufficient Evidence Guard in a TypeScript RAG pipeline?

Stage 1: Cosine Similarity Gate

- Evaluate the maximum similarity score among the retrieved `material_chunks` against `threshold = 0.70`.
- If `max_similarity < 0.70`, immediately classify the query as `INSUFFICIENT_EVIDENCE` without invoking the LLM.

Stage 2: LLM Verification Prompt

- For queries near the threshold (`0.65 - 0.75`), ask the fast model:
"Does the following excerpt directly address the question: [Question]? Output JSON: { is_supported: boolean }"
- If `is_supported` is false, return:
``evidence_state = 'INSUFFICIENT_EVIDENCE'``
``citations = []``
``content = 'I cannot find sufficient evidence in your uploaded study materials... '``

Write the TypeScript logic and provide a Vitest test file verifying that an off-topic question like 'How do I cook chicken biryani?' is rejected with 0 citations.

6. Adaptive Quiz & Rubric Assessment Prompts

Prompt 6.1: Strictly Grounded 4-Option MCQ Generation with Negative Constraints

Tool Used: Claude 3.7 Sonnet

Context: Prompt template in `lib/ai/quiz.ts`.

Goal: Generate strictly grounded MCQs with exactly 4 options, 1 correct answer, and zero metadata contamination.

Generate an adaptive assessment quiz based STRICTLY on the provided course material chunks.

TARGET CONCEPTS & DIFFICULTIES:

```
{{selected_concepts_with_target_difficulty_1_to_5}}
```

COURSE MATERIAL CHUNKS:

```
{{material_chunks_with_page_numbers}}
```

STRICT GENERATION CONSTRAINTS:

1. Every question MUST be directly answerable from the provided excerpts.
2. For multiple-choice questions (MCQ):
 - You MUST generate EXACTLY 4 distinct options.
 - Exactly ONE option must be mathematically/factually correct.
 - The remaining 3 options must be plausible distractors based on common misconceptions mentioned in the
 - Options must NOT contain "All of the above" or "None of the above".
3. Anti-Contamination Gate:
 - Do NOT include any concepts or questions about the software platform itself (e.g., 'PRD', 'Next.js',
 - Only test domain concepts from the uploaded document.
4. Output Schema:

Return valid JSON conforming to this schema:

```
{
  "questions": [
    {
      "concept_id": "string",
      "question_type": "mcq" | "open_ended",
      "difficulty": 1-5,
      "question_text": "string",
      "options": ["A", "B", "C", "D"], // exactly 4 for mcq; omit or empty for open_ended
      "correct_answer": "string",
      "explanation": "string with [Page X] citation"
    }
  ]
}
```

Prompt 6.2: Structured Open-Ended Rubric Grading Engine

Tool Used: Claude 3.7 Sonnet

Context: Prompt template in `lib/ai/grading.ts`.

Goal: Evaluate student essay/free-text explanations against objective rubric criteria.

You are an expert academic evaluator.
Grade the student's answer to the following open-ended question using a structured rubric.

QUESTION:
{{question_text}}

REFERENCE GROUND TRUTH (from course materials):
{{reference_explanation}}

STUDENT SUBMISSION:
{{student_answer}}

RUBRIC DIMENSIONS:

1. Factual Accuracy (0-100): Are the stated mechanisms and terminology correct?
2. Conceptual Depth (1-5): Did the student explain **why** and **how**, or merely state keywords?
3. Identified Strengths: Specific concepts the student grasped correctly.
4. Missing Concepts: Specific key elements or nuances omitted from the answer.
5. Constructive Feedback: A 2-sentence explanation of how to improve understanding.

Output valid JSON:

```
{
  "score": number (0-100),
  "is_correct": boolean (score ≥ 70),
  "conceptual_depth": number (1-5),
  "strengths": string[],
  "missing_concepts": string[],
  "feedback": "string"
}
```

7. Testing & Quality Assurance Prompts

Prompt 7.1: Inngest Background Worker Idempotency Test Suite

Tool Used: Claude 3.7 Sonnet

Context: Writing `inngest/material-processing.test.ts`.

Goal: Test failure handling, duplicate-job protection, and retry logic.

Write a comprehensive Vitest test suite for our Inngest `'material-processing'` background workflow (`'inngest`

Test Cases Required:

1. `'onFailure'` Handling:
 - When a job permanently fails (e.g. scanned PDF with no text), verifies that `'materials.status'` is updated to `'failed'`.
2. Duplicate-Job Protection (Idempotency):
 - When fired for a material already in `'processing'` status, immediately short-circuits to avoid duplicate jobs.
 - When fired for a material already in `'ready'` status, immediately exits without re-inserting chunks.
 - Asserts that firing the event twice results in exactly one set of chunks in `'material_chunks'`.
3. Plain Text & Markdown Uploads:
 - Verifies handling of `.docx` and `.txt` files, parsing them into page-mapped text chunks.

Provide full mock implementations for the Supabase client and Inngest step functions.

8. Production & Deployment Debugging Prompts

Prompt 8.1: Fixing PostgreSQL 42P01: relation "public.profiles" does not exist

Tool Used: Claude / Antigravity IDE Assistant

Context: Resolving migration failure in clean Supabase project.

Root Cause: Helper function `is_admin()` was defined before `profiles` table creation.

```
The initial Supabase database migration failed on a completely fresh project with:  
ERROR: 42P01: relation "public.profiles" does not exist  
LINE 25: (select is_admin from public.profiles where id = auth.uid())
```

Root cause:

```
`public.is_admin()` is being created BEFORE `public.profiles` exists.
```

Audit and rewrite the entire migration ``001_initial_schema.sql`` to enforce strict dependency order:

1. Enable ``pgvector`` extension first.
2. Generic triggers without table dependencies (``handle_updated_at``).
3. Create ``public.profiles`` table.
4. Create ``public.is_admin()`` function with ``SECURITY DEFINER`` and ``SET search_path = public``.
5. Create ``handle_new_user()`` function and trigger on ``auth.users``.
6. Create remaining 15 tables in order of foreign key dependencies (``spaces`` → ``projects`` → ``materials``).
7. Enable RLS and attach all 16 RLS policies.
8. Create ``match_chunks`` function.
9. Setup storage bucket and storage RLS policies.
10. Apply least-privilege security grants (revoke anon access, grant authenticated/service_role).

Do NOT change `vector(1536)`, table names, or RLS logic.

Prompt 8.2: Resolving Browser Failed to fetch During Signup

Tool Used: Antigravity IDE Assistant

Context: User encountered red banner "Failed to fetch" on `http://localhost:3000/signup`.

Root Cause: Placeholder Supabase credentials in `.env.local`.

```
The user attempted to sign up at http://localhost:3000/signup and received a red error banner: "Failed to  
In .env.local:
```

```
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
```

```
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key-here
```

```
SUPABASE_SERVICE_ROLE_KEY=your-service-role-key-here
```

The user has their Supabase project open in their browser dashboard.

Explain:

1. Exactly why "Failed to fetch" happens (browser attempts to connect to the dummy domain ``your-project.supabase.co``).
2. Give exact 3-click instructions for where to find their Project URL, anon key, and service_role key in the Supabase dashboard.
3. Remind them to disable "Confirm email" in Authentication → Providers → Email for direct local signin.
4. Restart the Next.js dev server so Turbopack inlines the real ``NEXT_PUBLIC_SUPABASE_URL``.
5. Verify connection using a Node.js script against the user's real Supabase URL without leaking secret keys.

9. Integrity Statement

All prompts documented herein were actively crafted, tested, and iterated upon during the design, development, and debugging of the **AI Study Companion** project. No hypothetical or unexercised prompts were included.